



UNIVERSITÄT
LEIPZIG

Software Engineering 2025/26 – Tutorium

Prof. Dr.-Ing. Norbert Siegmund
Tristan Scholz & Tobias Matzke



Tutorium 4 – Frameworks

- 1. Grundlagen
 - Was sind Frameworks?
 - Wofür braucht man Frameworks?
 - Framework vs. Library
- 2. Blueprint zur Einarbeitung in ein neues Framework
 - Mögliche Herangehensweise mit Schrittfolge
- 3. Praxis: Einarbeitung in Spring Boot
 - Anwendung der Blueprint
 - Grundbegriffe und kleines Beispielprojekt
- 4. Zusammenfassung/Ausblick

1. Grundlagen - Motivation

- Szenario: Entwicklung einer Web App
- Ausgangspunkt: leere index.html Datei
- Nötige Komponenten:
 - Template/Styling, Businesslogik
 - Routing-Mechanismus
 - Request Handling
 - Logging
 - Security
 - Session- und Cookie-Verwaltung
 - Datenbank-Anbindung
 - Authentifizierung/Rollenverwaltung
 - Error Handling
 - ...



Sehr viele generische Komponenten,
die man für jede Web App wieder
„neu“ entwickeln muss (= **AUFWAND**)

1. Grundlagen - Motivation

- Problem: generische, nicht-anwendungsspezifische, wiederkehrende Aufgaben
- Idee: **Grundgerüst** (aus Code) entwickeln, welches diese **generischen** Aufgaben erledigt
- **Grundgerüst** danach mit **anwendungsspezifischem** Code **ausfüllen**
- **Grundgerüst** wiederverwendbar für diverse Anwendungen



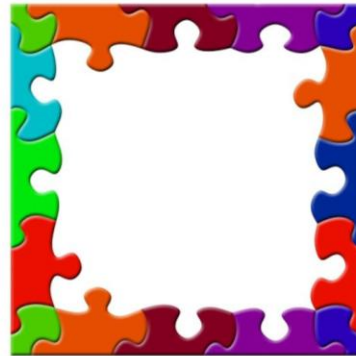
„Framework“

1. Grundlagen – Was ist ein Framework?

- **Framework = vorgefertigtes Gerüst** zur Entwicklung von Software
- stellt **grundlegende Strukturen** bereit, die genutzt und erweitert werden können
- Besteht aus (bereits fertigem) Code in einer bestimmten Programmiersprache
- Kann mit „eigenem“ Code einer Anwendung erweitert werden



„eigener Code“



Framework

→ Eigener Code muss sich an **Struktur** des Frameworks **anpassen**

1. Grundlagen – Was ist ein Framework?

- Szenario: Kind will Haus bauen, um damit zu spielen
 - Generische Fragen, die das Kind beantworten muss:
 - Welche Baumaterialien gibt es?
 - Wie verbindet man Bauteile?
 - Wie konstruiert man Türen, Fenster, ... ?
 - Wer kümmert sich um die Statik?
- hoher Zeitaufwand, außerdem ziemlich uninteressant
- Lösung: Kind sucht sich ein **Framework zum Hausbau**
- Framework „Lego“ liefert fertige, compatible Bausteine, welches das Kind nutzen kann, um eigene Projekte umzusetzen



1. Grundlagen – Pro und Contra



- Generischer Code muss nicht mehr selbst geschrieben werden
- Wiederverwendbarkeit
- Erprobter, optimierter Code
- Klare Struktur & Architektur



- Vorgegebene Struktur erfordert Anpassung → weniger flexibel
- Einstieg/Verständnis komplexer
- Abhängigkeit vom Framework

1. Grundlagen – Framework vs. Library

- Bibliothek = vorgefertigter Code, der generische, wiederkehrende Aufgaben lösen soll
 - Framework = vorgefertigter Code, der generische, wiederkehrende Aufgaben lösen soll
 - Unterschied: Kontrollfluss
 - Bibliothek: eigener Code liefert Grundstruktur, Bibliothek wird aus dem Code heraus aufgerufen
 - Framework: Framework liefert Grundstruktur, eigener Code wird in Framework-Struktur eingefügt
- „**Inversion of Control**“: Framework steuert den Ablauf und **ruft „deinen“ Code auf**, wenn es ihn „braucht“

2. Blueprint zur Einarbeitung in ein neues Framework

- Frage: Wie arbeitet man sich in eine neue Programmiersprache ein?
 - Tutorials, Einführung, Dokumentation anschauen
 - Syntax lernen
 - Hello World schreiben
 - Codebeispiele ausprobieren
 - Kleine Projekte ausdenken und umsetzen
 - Konzepte lernen (Objektorientierung)
- Unterschied bei Frameworks:
 - Zugrundeliegende Sprache muss verstanden werden
 - Komponenten/Struktur/Events verstehen
 - Weniger Low-Level-Details, sondern High-Level-Konzepte
 - Verstehen, wofür das Framework überhaupt gedacht ist

2. Blueprint zur Einarbeitung in ein neues Framework

- Schrittfolge:
 1. Orientierung
 2. „Fundament“ des Frameworks erfassen
 3. Überblick über das Framework und seine Konzepte
 4. Praktisches Anwenden
 5. Vertiefung und Experimentieren
 6. Reflexion und Festigung
- **Ausführliches Dokument mit Schrittfolge + Hinweisen: siehe Moodle**

2. Blueprint zur Einarbeitung in ein neues Framework

- Schritt 1: Orientierung
 - Ziel: *Ordne ein, warum das Framework existiert, was für Probleme es löst und wofür du es lernen möchtest.*
- Schritt 2: „Fundament“ des Frameworks erfassen
 - Ziel: *Lerne die Grundlagen der Programmiersprache, auf der das Framework aufbaut, und versuche, wichtige Konzepte/Paradigmen der Sprache zu verstehen.*
- Schritt 3: Überblick über das Framework und seine Konzepte
 - Ziel: *Begreife den grundlegenden Aufbau des Frameworks und zentrale Kernbegriffe, die das Framework nutzt (und es einzigartig machen).*

2. Blueprint zur Einarbeitung in ein neues Framework

- Schritt 4: Praktisches Anwenden
 - *Ziel: Erarbeite dir ein funktionales Verständnis des Frameworks, indem du erste kleine Minimalbeispiele selbst codest.*
- Schritt 5: Vertiefung und Experimentieren
 - *Ziel: Probiere dich an fortgeschrittenen Themen und größeren Projekten, um die Mechanismen des Frameworks im Detail zu begreifen.*
- Schritt 6: Reflexion und Festigung
 - *Ziel: Reflektiere dein Wissen UND deine Wissenslücken regelmäßig, um das Gelernte zu festigen und dir neue Fähigkeiten anzueignen*

Anwendungsteil:

Einarbeitung in Spring Boot anhand der Blueprint

Was ist Spring Boot?

- Open-Source Java Framework
- Basiert auf Spring
- Vor allem zur Entwicklung von Micro Services und Webanwendungen genutzt
- Erleichtert sonst umständliche Konfigurationen, bietet integrierten Webserver, ...

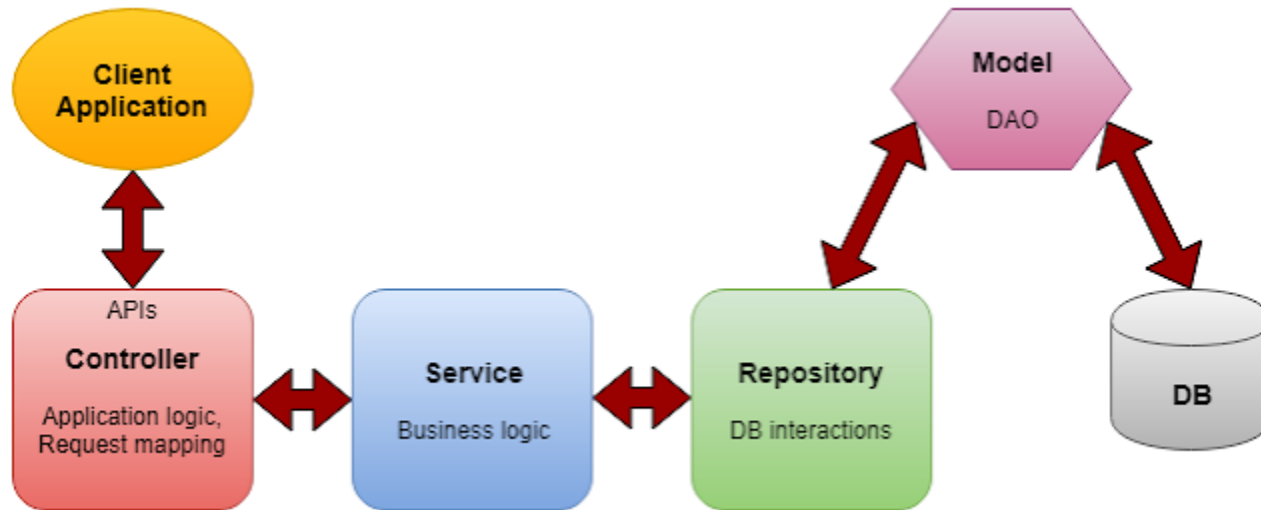


Unterliegende Konzepte

- Programmiersprache: Java (oder Kotlin, Groovy)
 - Objektorientierte Konzepte
- REST-API & HTTP
 - APIs als Kommunikationsschnittstellen zwischen Frontend und Backend / Datenbank
 - Anfragetypen: GET, POST, PUT, DELETE, (PATCH), ...
- JSON zum Objekttransfer
 - Bsp. JSON-Studierendenobjekt: {"id": 1, "name": "Max Mustermann"}
- Build-Tools (Maven, Gradle) zum Dependency-Management

Grundkonzepte Spring Boot

- Grundlegende Architektur:



<http://randikatech.blogspot.com/2019/09/get-your-hands-dirty-with-micro-services.html>

Grundkonzepte Spring Boot

- Projekterstellung über spring initializr (<https://start.spring.io/>)
- Spring Annotations
 - Nutzung von Annotations (z.B. @Service, @Repository) zur Verwaltung von Klassen & Methoden
- Dependency Injection & Inversion of Control
 - Abhängigkeiten einer Klasse automatisch von Spring bereitgestellt
 - Klasse muss diese nicht selbst erstellen

Praktisches Beispiel

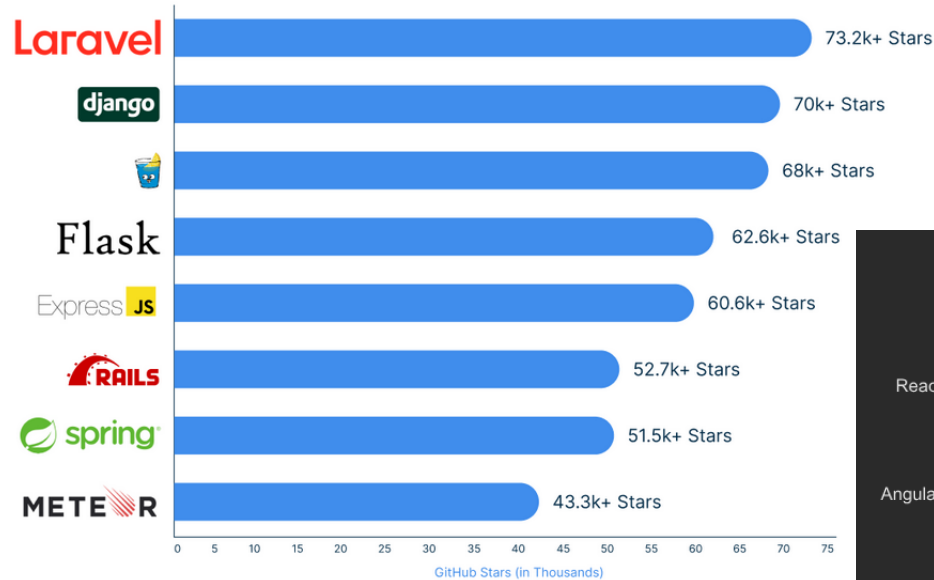
- Basierend auf:
 - Spring Quickstart (<https://spring.io/quickstart>)
 - Geeksforgeeks Spring Boot Tutorial (<https://www.geeksforgeeks.org/advance-java/spring-boot/>)
 - Spring Boot Tutorial Amigoscode (<https://www.youtube.com/watch?v=9SGDpanrc8U&t>)
- Fertiger Code im [GitLab](#)

4. Zusammenfassung/Ausblick

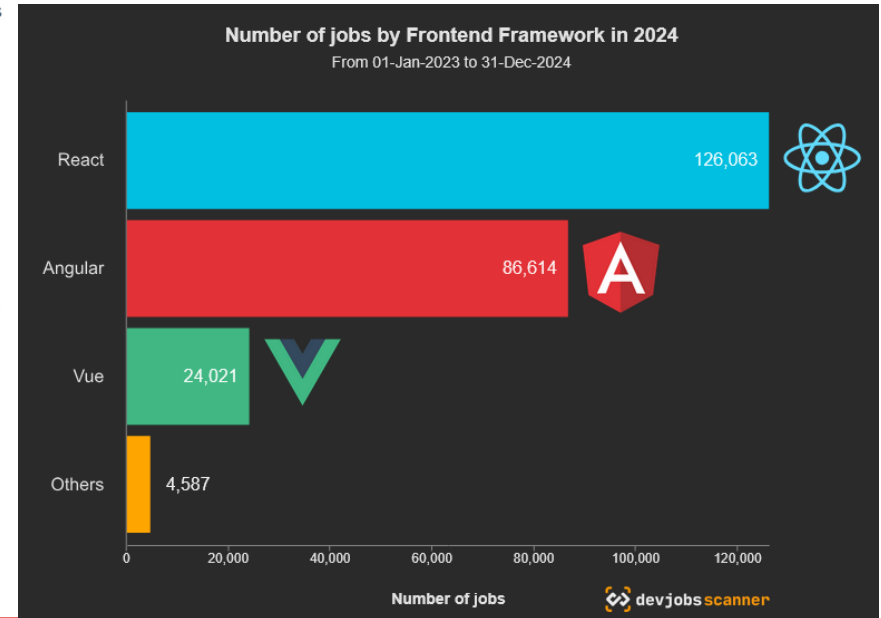
Wiederholung

- Framework = vorgefertigtes Gerüst zur Entwicklung von Software
- Übernimmt wiederkehrende, generische Aufgaben
- Inversion of Control: Framework steuert/strukturiert Ablauf und ruft Anwendungscode auf
- Hinweise zum Lernprozess:
 - Lernen = Individueller Prozess
 - Strukturierter Lernansatz hilfreich
 - Nötiges Basiswissen/Fundament früh lernen, Begriffsübersichten erstellen
 - **Früh Code selbst schreiben (ohne KI-Unterstützung!!)**
 - Best Practices von Beginn an beachten

6 Most Popular Backend Frameworks to Use in 2025



<https://www.mindinventory.com/blog/best-backend-frameworks/>



<https://www.devjobsscanner.com/blog/the-most-demanded-frontend-frameworks/>

Praktische Bedeutung

- Absoluter Industriestandard in Unternehmen
 - Frameworks lernen = essentieller Core Skill für Software Developer
 - (besser 2-3 Frameworks richtig verstehen als 10 nur oberflächlich)
 - Wissen über Framework A häufig auf Framework B übertragbar
 - Extrem viele Frameworks für verschiedene Bereiche verfügbar:
 - Webframeworks (Frontend, Backend), Testframeworks, Application Frameworks, Gameplay Frameworks, ...
- **Frameworks = Unverzichtbares Grundwerkzeug moderner Softwareentwicklung und „Eintrittskarte“ in professionelles, strukturiertes Arbeiten**



UNIVERSITÄT
LEIPZIG

Fragen?

Tristan Scholz: scholz@studserv.uni-leipzig.de

Tobias Matzke: matzke@studserv.uni-leipzig.de